

Magnetic Passage Logger

Commit accepted evidence before presenting a count

Freeze, timestamp, persist, recover, then display

Lesson	036 — project	Time	180–240 minutes
Prerequisites	Lessons 010, 022, 034, and 035	Board	Mega fixture
Evidence	three LEDs, one digit, two host slots	Status	host verified; bench open
Energy class	E0 core and persistence fixtures; provisional E1 indicators		

Scope gate. The project consumes accepted software records. It does not resample sensors, decode an encoder, implement a filesystem, move a gate, identify a person, or actuate a load. Its RTC, byte storage, and passage inputs are deterministic host fixtures. Physical RTC, removable media, magnetic specimens, and persistence acceptance remain open.

1 Mission and durable evidence

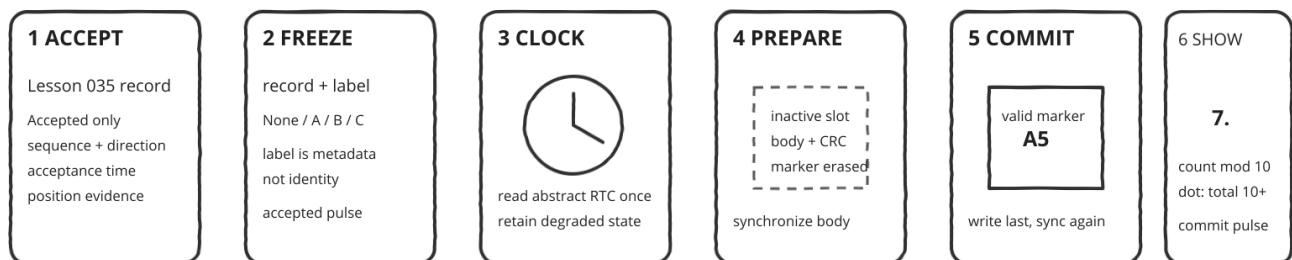
The project preserves:

accepted passage → frozen clock and label → durable commit → presented count.

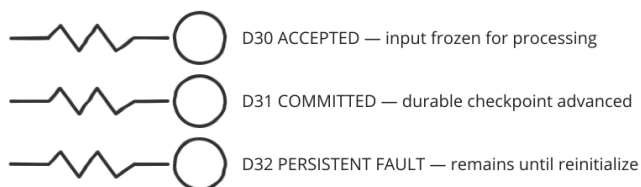
A label **None/A/B/C** is local metadata, never identity, authentication, access authority, or a safety decision. A degraded **ClockReading** remains degraded; the logger never calls it a valid timestamp.

2 Build one commit in visible stages

ONE ACCEPTED RECORD BECOMES ONE DURABLE COMMIT



VISIBLE PROGRESS AND FAILURE BOUNDARIES



Healthy update: D30 + D31 together. Inject fixture failure to separate stages.

RTC read fails

pending record and label remain frozen;
retry same sequence and read clock again.

newer record while pending

retain one bounded overrunInput;
do not queue or silently commit it.

Display failure never rewrites durable state.

Alternative text: six pencil stages freeze one accepted record and local label, read the abstract clock, prepare and synchronize an inactive slot, write the valid marker last, then present the committed count; side paths show RTC retry, bounded overrun, and three evidence LEDs.

The dependent progression is visible. D30 means an accepted record was frozen; D31 means the ledger advanced its durable checkpoint. On an immediate successful update both pulses are true in the same snapshot. To observe the stages separately, deliberately fail the RTC or ledger in the deterministic fixture: D30 appears without D31, then a same-sequence retry can produce D31. D32 holds a persistent fault until successful shutdown and reinitialization. The one-digit display changes only after commit: it shows `committedCount` mod 10, with decimal point meaning ten or more.

Evidence path	Fixture role	Meaning
D30 through 1 kΩ	accepted pulse	record accepted for processing
D31 through 1 kΩ	committed pulse	durable checkpoint advanced
D32 through 1 kΩ	persistent fault	retry/recovery/presentation fault
D22/D23/D24	74HC595 data/clock/latch	committed count presentation only
D28	label button	previews <code>-/A/b/C</code> , then restores durable count

These provisional roles are not a sensor or storage schematic. The known indicator/segment bound is 55 mA: three direct LEDs at no more than 5 mA each plus Lesson 010's 40 mA segment total. It excludes the Mega baseline, shift register package current, sensor modules, and pulls.

3 Logger contract

`MagneticPassageLogger` borrows an abstract `Rtc` and one `PassageLedger`. It presents the durable count through the narrow `PassageCountDisplay` boundary. A lesson-local seven-segment adapter binds the inert display to that boundary. Initialization validates configuration, initializes ledger then RTC then display, recovers a checkpoint, and rolls back in reverse order. Shutdown is idempotent.

`LoggedPassage` retains accepted sequence, resulting committed count, acceptance time, direction, frozen label, copied position evidence, complete clock reading, and explicit sequence-gap evidence.

Snapshot field	Meaning
<code>selectedLabel</code>	metadata selected for the next new sequence
<code>committedCount/Sequence</code>	recovered or newly durable authority
<code>pending, pendingInput</code>	frozen input awaiting clock or ledger
<code>overrun, overrunInput</code>	one bounded newer input, not queued
<code>hasFrozenEntry, frozenEntry</code>	immutable bytes after RTC success
<code>acceptedPulse/committedPulse</code>	one-call evidence intents
<code>persistentFault</code>	latched fault intent
<code>recovery, presentationStatus, status</code>	explicit outcomes

Only an `Accepted` Lesson 035 record is consumable. An older sequence is rejected. An equal sequence is idempotent only when direction, acceptance time, and position agree with recovered durable evidence; disagreement is an invariant fault. A forward gap may commit, but `sequenceGap` records it.

4 Clock, retry, and overrun

For a new sequence, the logger freezes record and current label, then attempts one RTC read per update. A failed read emits accepted evidence, performs no ledger or display work, and leaves the input pending.

Retrying the same sequence reads the RTC again. The first successful clock reading becomes part of an immutable frozen entry and is never reread during ledger retries.

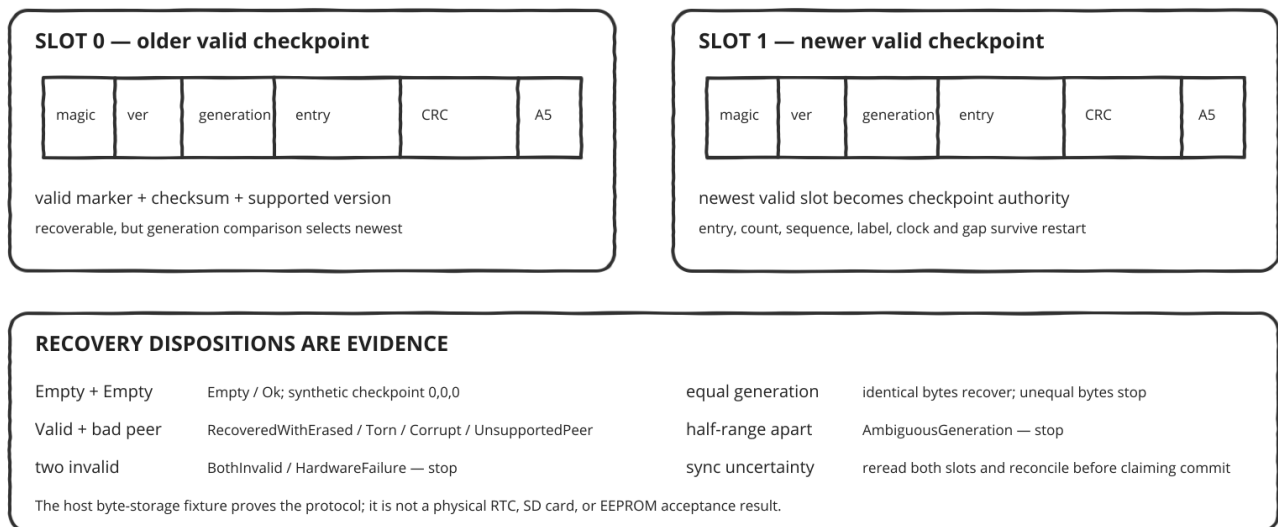
A failed ledger commit changes no committed count, sequence, display, or generation. Retrying the same sequence uses byte-identical entry data. While pending, one newer record becomes bounded `overrunInput`; later arrivals replace that evidence. It is neither queued nor silently committed.

Presentation happens after durability. Display failure cannot change committed state or trigger a ledger retry; it records presentation status and latches the fault witness. A simultaneous label-button edge applies only to the next record because the pending label was already frozen.

The canonical sketch makes label selection board-visible without confusing it with durable authority. A D28 press briefly previews `-, A, b, or C` for `None/A/B/C`; after one second it restores the committed count and decimal-point convention. The sketch preserves the pre-cycle D30/D31 pulse intents, checks `cycleLabel()`, refreshes the post-cycle snapshot, and never treats the temporary glyph as a committed count.

5 Two-slot recovery

RECOVER DURABLE EVIDENCE — NEVER GUESS BETWEEN AMBIGUOUS SLOTS



Alternative text: pencil drawings compare two alternating fixed-size ledger slots; the newest supported valid slot wins, while erased, torn, corrupt, duplicate-generation, unsupported-version, and ambiguous generation states remain explicit recovery evidence.

Each fixed-size slot stores magic, version, length, modular generation, the fully encoded entry, resulting count and sequence, IEEE CRC-32, and valid marker. Integer fields are explicitly little-endian; no C++ object memory image is stored. Commit invalidates the inactive marker, writes body and checksum in order, synchronizes, writes marker `0xa5` last, then synchronizes again.

Two erased slots recover as Empty with synthetic checkpoint `{0,0,0}`. One valid supported slot wins over an erased, torn, corrupt, or unsupported peer while preserving that disposition. Two invalid non-erased slots stop. Equal generations recover only when bytes are identical. Unequal duplicate generations and exact half-range separation are ambiguous and stop.

When synchronization fails, the ledger rereads both slots. It reports committed after reconciliation only when the new checkpoint is provably newest; reports not committed when the old checkpoint still wins; and becomes unusable when the result or reread is ambiguous.

6 Narrative example and evidence order

```
void loop ()
{
    observeAcceptedPassage ();
    decideLabelAndCommit ();

    if (!actuateDurableEvidence ())
    {
        stopSafely ();
    }
}
```

The full fixture acquires indicator outputs and display, configures the two-slot ledger, RTC fixture, and logger, then starts with recovered state visible. Observe–decide–actuate order never makes presentation authoritative over persistence.

7 Predict, observe, interpret

1. Predict Empty recovery and displayed zero from two erased host slots.
2. Submit one accepted record. Predict D30 and D31 together when clock, ledger, and presentation all succeed in the same call.
3. Deliberately fail the RTC or ledger fixture. Predict D30 without D31, then retry the same sequence and observe D31 only after durable commit.
4. Fail the first RTC read. Predict frozen pending input, no ledger write, then one new clock read on same-sequence retry.
5. Fail a body write or synchronization. Predict no logical advance unless reread proves the new slot newest.
6. Present a newer record while pending. Predict one overrun snapshot and no hidden queue.
7. Corrupt the inactive peer and restart. Predict recovered durable entry plus explicit corrupt-peer disposition.
8. In the host fixture, fail display presentation. Predict durable count unchanged and persistent fault. The canonical Mega sketch does not inject a display failure; it visibly demonstrates the RTC retry only.

8 Diagnosis and exercises

Observation	Interpret and inspect
D30 without D31	accepted input is pending clock or ledger completion
D31 but display stale	durable commit succeeded; presentation failed
count changes before D31	reject the fixture; presentation preceded authority
restart chooses older slot	inspect generation, marker, length, CRC, version
two slots disagree equally	stop; do not guess a winner
new input disappears	inspect bounded overrun evidence and caller policy

1. Explain why the valid marker is written last.
2. Trace an RTC failure followed by ledger failure and successful retry.
3. Distinguish `RecoveredWithTornPeer` from `BothInvalid`.
4. Explain why label and clock are excluded from equal-sequence comparison.
5. Show why display failure cannot justify rewriting a ledger slot.

Stop rule. Use only deterministic RTC, byte-storage, and accepted-passage fixtures plus the inert indicator panel. Remove USB for heat, odor, reset, unstable rail, unexpected display/LED state, or current-limit trip. Do not connect unknown sensors, physical RTC/media, motion, gates, locks, vehicles, launchers, ignition paths, or safety systems.

9 Software evidence

Host tests must cover directions and labels, recovery dispositions, every byte/sync failure, byte-identical retry, torn restart, generation/sequence edges, pending overrun, lifecycle rollback, display failure, and deterministic trace/state bytes. Compilation, replay, this PDF, and host-slot recovery are not physical persistence, RTC, sensor, or bench evidence. The searchable HTML lesson is the primary accessible route. This PDF remains untagged and makes no PDF/UA claim.

The Mega fixture deliberately fails its first abstract RTC read. D30 and D32 make the pending fault visible without D31; the same-sequence retry then commits, and a bounded two-second persistent-fault interval ends with shutdown/reinitialization and recovery from the retained software slots. The D28 pull-up button cycles **None/A/B/C**; its edge is applied after the current logger update, so a simultaneous press affects only the next new sequence. Its bounded one-second **-/A/b/C** preview then restores the durable count.

10 What the result establishes

A passing fixture establishes deterministic policy: accepted data freezes before clock and ledger work; retries preserve the right bytes; a durable checkpoint precedes presentation; recovery never silently chooses ambiguous evidence; and bounded overrun remains visible. It does not establish EEPROM or SD wear life, RTC accuracy, power-loss behavior on a physical board, sensor identity, passage geometry, current margin, or suitability for access control.

Authority	Consult it for
<code>magnetic_passage_logger.h</code>	lifecycle, pending, pulse, and status contract
<code>passage_ledger.h</code>	slot recovery and commit interface
focused host tests	failure, retry, recovery, and deterministic byte evidence
Lessons 034–036 design record	frozen policy and deferred physical gates
testing and safety contracts	evidence language, current limits, and stop rules

The HTML lesson remains the searchable companion. Exact physical components need primary sources, an authoritative schematic, current accounting, and a recorded human bench result before any support or persistence claim.